



Quality Planning & Goal Setting

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

The Quality Engineering Process

Quality Engineering is a disciplined approach that integrates quality goals, processes, planning, and evaluation throughout the software development life cycle.

Key Quality Engineering Activities

- **Quality Planning:** Setting target quality goals and forming a validation strategy for the project.
- **Quality Assessment:** Measuring software characteristics against benchmarks during development.
- **Quality Improvement:** Feedback loops to tune the development process, preventing recurring code defects.
- **Process Integration:** Ensuring quality tasks are built into milestones (requirements, architecture, code, test).

Quality Planning & Strategy

- **Defining Objectives:** Agreeing on target benchmarks (e.g. max 0.5 bugs/KLOC, 99.9% availability).
- **Selecting QA Methods:** Deciding where to use inspections vs dynamic unit testing or load testing.
- **Resource Budgeting:** Allocating developers, QA testers, and hardware environments for testing.
- **Risk Assessment:** Identifying critical modules needing rigorous verification (e.g. payment APIs).

Goal-Question-Metric (GQM) Paradigm

GQM is a structured framework developed by Basili to establish measurable software goals:

Hierarchical GQM Structure

- 1. Goal (Conceptual Level):** State the objective for a project, process, or product. Must define object, purpose, focus, viewpoint, and context.
- 2. Question (Operational Level):** Formulate questions that determine if the goal is being met during development.
- 3. Metric (Quantitative Level):** Define specific, numeric metrics to answer each question objectively.

GQM Example in Practice

- **Goal:** Improve the reliability of the billing API from the developer's viewpoint.
- **Questions to ask:**
 - Q1: What is the current crash frequency?
 - Q2: Are exception blocks trapping failures?
- **Metrics defined:**
 - M1: Mean Time Between Failures (MTBF) in hours.
 - M2: Percentage of database errors logged vs. uncaught runtime exceptions.

Quality Assessment & Improvement

How companies measure quality during development and perform improvements:

Quality Assessment (Measurement)

- **Metrics Collection:** Collecting quantitative data on defect density, test coverage, and compiler warnings during the project.
- **Benchmarking:** Comparing metrics against standards (e.g., requiring 80% code coverage before a pull request can be merged).
- **Quality Audits:** Periodic reviews by QA leads to verify code matches design guidelines.

Continuous Quality Improvement

- **Root Cause Analysis (RCA):** When a severe defect escapes to production, the team conducts an RCA to identify the underlying process failure.
- **Retrospectives:** Post-sprint review sessions where developers identify bottlenecks and update quality checklists.
- **Process Tuning:** Modifying design constraints, compiler rules, or unit test targets based on RCA feedback, preventing repeat bugs.

Applications of Quality Planning

Software teams implement GQM and quality plans across various development domains:

Agile Sprint Planning

Teams establish Definition of Done (DoD) goals at sprint start. E.g., 'A story is Done only if it has 80% test coverage and passes security scans (DoD Goal).'

Medical Device Verification

Quality plans detail the specific test runs, compliance audits, and safety-critical thresholds required to meet FDA validation criteria.

Cloud Microservices Scaling

Applies GQM to monitor system latency. Goal: Optimize query speeds. Metrics tracked: 99th percentile response time, CPU utilization under load.

Legacy Refactoring Projects

Establishes quality plans to minimize regression bugs during migrations. Defines strict validation thresholds prior to production deployment.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management